

Clasificación de Tumores Cerebrales en MRI mediante Redes Neuronales Convolucionales propias sobre el Dataset *Brain Tumor Classification MRI*

Abel Albuez Sánchez, Daniel Rios, Juan Torres y Javier Esquivel
Maestría en Analítica para Inteligencia de Negocios (MAIN)
Pontificia Universidad Javeriana, Bogotá D.C., Colombia
Profesor: Julio Omar Palacio Niño, M.Sc. Aprendizaje Profundo 2026

Resumen—La clasificación automática de tumores cerebrales en imágenes de resonancia magnética (MRI) constituye una tarea relevante para el apoyo al diagnóstico radiológico. Este trabajo aborda la clasificación de cuatro categorías —*glioma*, *meningioma*, *pituitary* y *no tumor*— sobre el dataset público *Brain Tumor Classification MRI* de Kaggle (3 264 imágenes), mediante tres arquitecturas propias de redes neuronales convolucionales (CNN) entrenadas desde cero, sin emplear *transfer learning*. Se reporta un análisis exploratorio detallado del dataset —incluyendo distribución por clases, dimensionalidad y estadísticos de intensidad—, un pipeline de preprocesamiento con normalización tipo ImageNet y *data augmentation*, y un protocolo de evaluación basado en *F1-Score* macro, adecuado para escenarios con desbalance de clases.

Index Terms—clasificación de tumores cerebrales, redes neuronales convolucionales, resonancia magnética, aprendizaje profundo, PyTorch, *F1-Score* macro

I. OBJETIVO DEL PROYECTO

I-A. Contexto Clínico

El diagnóstico temprano y preciso de tumores cerebrales constituye uno de los retos más relevantes de la neuro-oncología contemporánea. La resonancia magnética (MRI, *Magnetic Resonance Imaging*) es la modalidad de imagen de referencia para la detección y caracterización de estas lesiones; sin embargo, su interpretación depende fuertemente de la experiencia del radiólogo y está sujeta a variabilidad inter-observador. En este contexto, los sistemas de apoyo al diagnóstico basados en aprendizaje profundo han mostrado resultados prometedores como herramientas de cribado y segunda opinión, capaces de procesar grandes volúmenes de estudios de manera consistente.

I-B. Objetivo Técnico

El presente proyecto tiene como objetivo diseñar, entrenar y evaluar tres arquitecturas propias de redes neuronales convolucionales (CNN) para la clasificación automática de imágenes MRI cerebrales en cuatro categorías: *glioma_tumor*, *meningioma_tumor*, *no_tumor* y *pituitary_tumor*. Se utiliza el dataset público *Brain Tumor Classification MRI* disponible en Kaggle. De manera explícita, este trabajo **no** emplea técnicas de *transfer learning* ni pesos preentrenados sobre ImageNet u otros corpus; el énfasis pedagógico está en la construcción de arquitecturas convolucionales desde cero y en la comprensión del impacto de sus hiperparámetros y bloques constitutivos.

Cuadro I: Distribución de imágenes por clase y partición.

Clase	Train	Test	Total
glioma_tumor	826	100	926
meningioma_tumor	822	115	937
no_tumor	395	105	500
pituitary_tumor	827	74	901
Total	2 870	394	3 264

I-C. Criterios de Evaluación

La evaluación de los modelos se realizará mediante un conjunto de métricas clásicas de clasificación multiclase: *Accuracy*, *Precision*, *Recall* y *F1-Score* en su variante *macro-averaged*, complementadas con la matriz de confusión por modelo. Dado el desbalance entre clases observado durante el análisis exploratorio (Sección II), el *F1-macro* se adopta como métrica principal de comparación, por su robustez frente a clases minoritarias.

II. ANÁLISIS DEL DATASET Y DIMENSIONALIDAD

II-A. Descripción del Dataset

El conjunto de datos *Brain Tumor Classification MRI*, publicado en Kaggle, contiene **3 264 imágenes** de resonancia magnética cerebral organizadas en cuatro clases diagnósticas: *glioma_tumor*, *meningioma_tumor*, *pituitary_tumor* y *no_tumor* (estudios sin hallazgos tumorales). Los autores del dataset proporcionan una partición oficial entre conjuntos de entrenamiento y prueba que se respeta en este trabajo. La Tabla I resume su distribución.

El conjunto de entrenamiento concentra el **87,9 %** de las imágenes (2 870), mientras que el conjunto de prueba representa el **12,1 %** restante (394 imágenes). El conjunto *Testing* se reserva como *holdout* independiente para la evaluación final de los modelos, y la subdivisión interna para validación se realiza únicamente sobre *Training* (véase Sección IV-B).

II-B. Balance de Clases

El análisis de balance evidencia que la clase *no_tumor* es la minoritaria en entrenamiento, con aproximadamente **395** imágenes, frente a **~825** para cada una de las tres clases tumorales. Esta proporción asimétrica ($\sim 1:2$) puede inducir

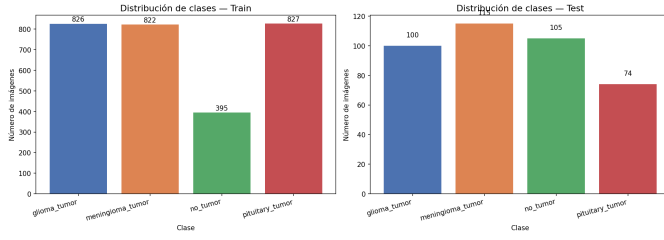


Figura 1: Distribución absoluta por clase y partición.

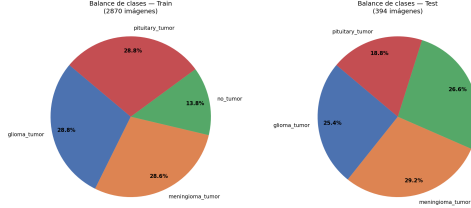


Figura 2: Proporción relativa en entrenamiento.

sesgos durante el aprendizaje, llevando al modelo a sub-representar la clase minoritaria y a inflar artificialmente la *Accuracy* global.

Como estrategia de compensación se adopta el uso de pesos por clase (*class_weight*) en la función de pérdida *Cross-Entropy*, con ponderaciones inversamente proporcionales a la frecuencia relativa de cada clase en el conjunto de entrenamiento. Esto incrementa la magnitud del gradiente para errores cometidos sobre la clase minoritaria sin alterar la distribución de muestras observadas. Las Figuras 1 y 2 ilustran gráficamente esta distribución.

II-C. Análisis de Dimensionalidad

Se inspeccionó una muestra de **80 imágenes por clase** para caracterizar la variabilidad en resolución espacial del dataset. Las imágenes presentan resoluciones altamente heterogéneas: las dimensiones máximas observadas alcanzan **1024×768 px** y las mínimas **59×80 px**. La relación de aspecto también varía considerablemente entre estudios, reflejo de las diferencias entre equipos de adquisición y protocolos clínicos.

Dada esta heterogeneidad, se establece el redimensionamiento uniforme a **224×224 px** como paso obligatorio del pipeline. Esta resolución constituye un compromiso adecuado entre el nivel de detalle anatómico preservado y el costo computacional, y es además el formato canónico de las arquitecturas CNN modernas, lo que facilita comparaciones posteriores. Las Figuras 3 y 4 muestran respectivamente el histograma de resoluciones y la distribución de relaciones de aspecto observadas por clase.

II-D. Análisis Estadístico de Intensidades

Se analizaron las distribuciones de intensidad de píxeles (escala de grises) y canales RGB por clase, con el fin de identificar diferencias fotométricas potencialmente discriminativas para el modelado. La Tabla II resume los estadísticos descriptivos.

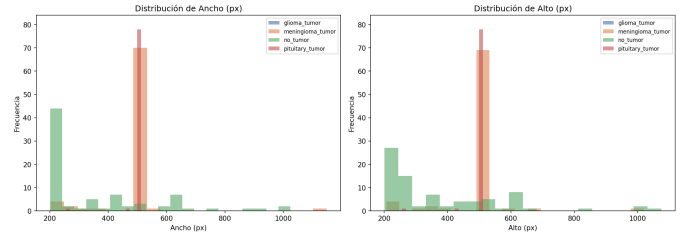


Figura 3: Histograma de resoluciones (ancho × alto) en el dataset.

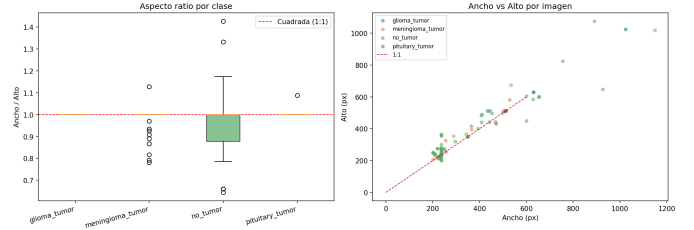


Figura 4: Distribución de relaciones de aspecto por clase.

Cuadro II: Estadísticas de intensidad (escala 0–255) por clase.

Clase	Media	Std	Mín	Máx	Mediana
glioma_tumor	36,23	41,43	0	255	14
meningioma_tumor	44,56	47,77	0	255	28
no_tumor	52,21	59,63	0	255	22
pituitary_tumor	49,50	42,94	0	255	50

Los *gliomas* presentan los valores más bajos de media (36,23) y mediana (14), lo que los caracteriza como tejidos relativamente **hipointensos** en la modalidad analizada. Los tumores *pituitarios*, en contraste, presentan la mediana más alta (50) y la menor dispersión relativa, indicando intensidades más concentradas y homogéneas. La clase *no_tumor* muestra la mayor desviación estándar (59,63), consistente con la variedad anatómica del cerebro sano sin focalización lesional. Estas diferencias fotométricas constituyen señales útiles que una CNN puede explotar a nivel de filtros convolucionales de bajo nivel.

Para estabilizar el entrenamiento se aplicará normalización *Z-score* sobre los tres canales utilizando los estadísticos canónicos de ImageNet ($\mu = [0,485, 0,456, 0,406]$, $\sigma = [0,229, 0,224, 0,225]$), valores ampliamente adoptados en la literatura como referencia robusta incluso cuando no se emplea *transfer learning*. Las Figuras 5 y 6 complementan visualmente este análisis.

II-E. Imagen Promedio y Muestras por Clase

La Figura 7 presenta la imagen promedio por clase, calculada como la media píxel a píxel de 100 imágenes redimensionadas a 224×224 px. Los gliomas exhiben regiones **hipointensas difusas**, de bordes poco definidos, consistente con su naturaleza infiltrativa. Los tumores *pituitarios* tienden a concentrarse en la zona central, reflejando la ubicación anatómica de la glándula hipófisis. Los meningiomas muestran

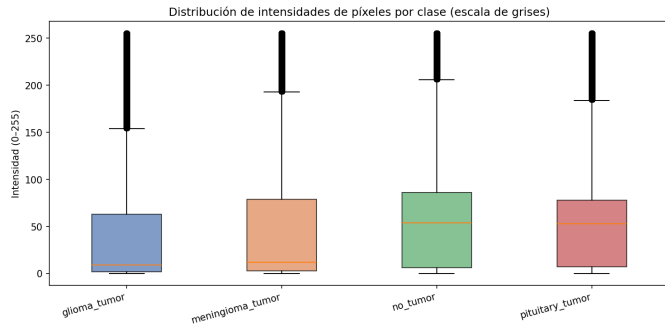


Figura 5: Diagramas de caja de intensidades de píxel por clase.

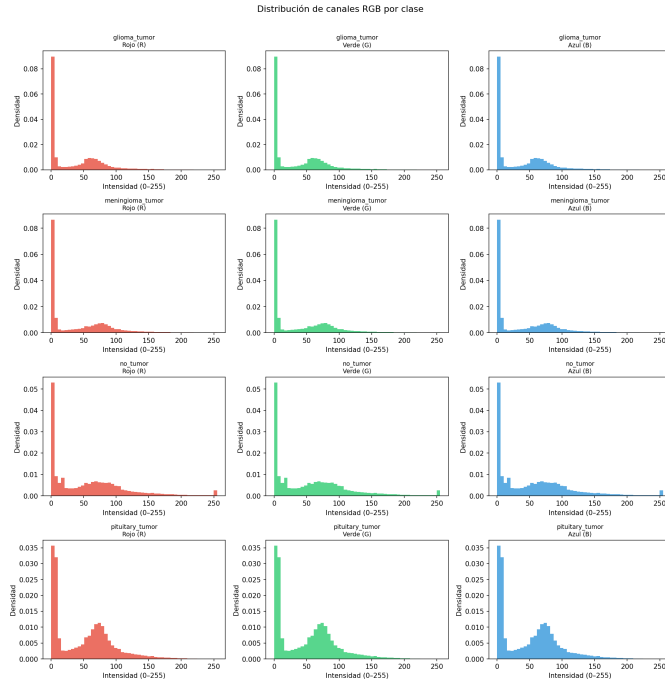


Figura 6: Distribución de intensidades por canal RGB y por clase.

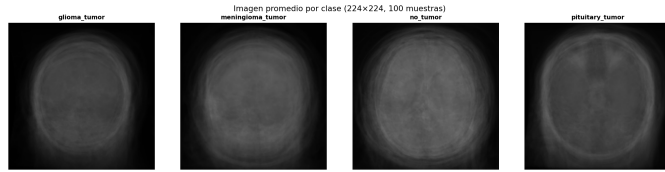


Figura 7: Imagen promedio por clase (100 imgs, 224×224 px).

patrones intermedios con tendencia a localizarse en la periferia. La clase *no_tumor*, al promediar tejidos cerebrales sanos en distintas orientaciones, produce una imagen más difusa y centralmente simétrica.

La Figura 8 ilustra cinco ejemplos representativos por clase. Se evidencia alta variabilidad intra-clase en orientación, escala y contraste, lo que justifica el uso de *data augmentation* — rotación $\pm 15^\circ$, *flip* horizontal y variación de brillo— durante el entrenamiento para mejorar la capacidad de generalización

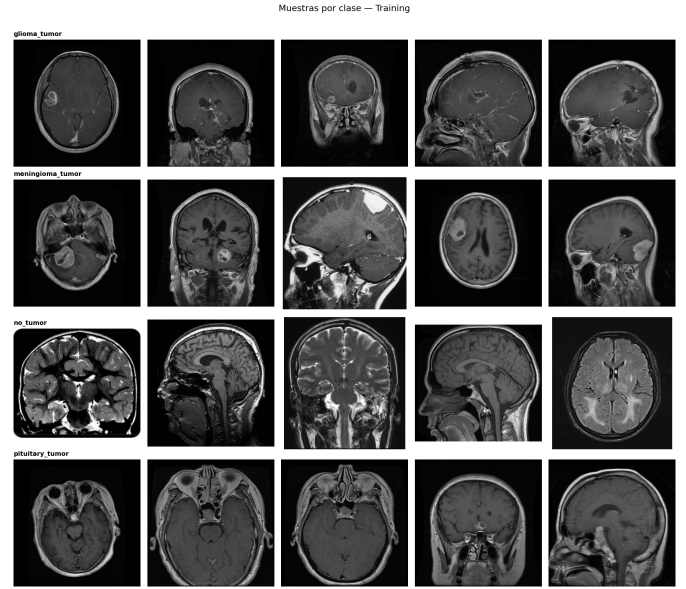


Figura 8: 5 muestras por clase — *Training*.

Cuadro III: Pipeline de preprocesamiento propuesto.

#	Paso	Justificación
1	Resize 224×224 px	Resoluciones heterogéneas requieren entrada uniforme.
2	Conversión a RGB	Algunas imágenes en escala de grises; se unifica formato.
3	Z-score (μ ImageNet)	Estabiliza el gradiente y mejora la convergencia.
4	<i>Data augmentation</i>	Rotación $\pm 15^\circ$, <i>flip</i> horizontal y variación de brillo.
5	<i>class_weight</i>	Compensa el desbalance de <i>no_tumor</i> .
6	Split 80/20 <i>Training</i>	Separa validación sin contaminar el <i>holdout</i> oficial.

del modelo.

II-F. Plan de Preprocesamiento

A partir de los hallazgos anteriores, se define el pipeline de preprocesamiento resumido en la Tabla III, aplicado de forma consistente a las tres arquitecturas evaluadas.

III. METODOLOGÍA

III-A. Métricas de Evaluación

Para evaluar las arquitecturas propuestas se emplea un conjunto de métricas estándar de clasificación multiclase:

- **Accuracy:** proporción global de aciertos, $\text{Acc} = (TP + TN) / (TP + TN + FP + FN)$.
- **Precision (macro):** promedio no ponderado de la precisión por clase, $\frac{1}{C} \sum_c TP_c / (TP_c + FP_c)$.
- **Recall (macro):** promedio no ponderado de la sensibilidad por clase, $\frac{1}{C} \sum_c TP_c / (TP_c + FN_c)$.
- **F1-Score (macro):** media armónica de Precision y Recall, $F1 = 2 \cdot P \cdot R / (P + R)$, calculada por clase y promediada de forma no ponderada.
- **Matriz de confusión:** análisis cualitativo de los errores entre pares de clases.

Cuadro IV: Arquitectura del Modelo 1 (CNN Baseline). Entrada ($B, 3, 224, 224$).

#	Capa	Filt.	Kernel	Stride	Salida (C, H, W)	Params
1	Conv2D	16	3×3	1	(16, 224, 224)	448
2	ReLU	—	—	—	(16, 224, 224)	0
3	MaxPool2D	—	2×2	2	(16, 112, 112)	0
4	Conv2D	32	3×3	1	(32, 112, 112)	4 640
5	ReLU	—	—	—	(32, 112, 112)	0
6	MaxPool2D	—	2×2	2	(32, 56, 56)	0
7	Conv2D	64	3×3	1	(64, 56, 56)	18 496
8	ReLU	—	—	—	(64, 56, 56)	0
9	MaxPool2D	—	2×2	2	(64, 28, 28)	0
10	Flatten	—	—	—	(50 176,)	0
11	Linear	—	—	—	(128,)	6 422 656
12	ReLU	—	—	—	(128,)	0
13	Dropout(0.5)	—	—	—	(128,)	0
14	Linear	—	—	—	(4,)	516
Total parámetros entrenables:						6 446 756

Dado el desbalance descrito en la Sección II, la *Accuracy* puede ser engañosa, pues un modelo que ignore por completo la clase *no_tumor* aún alcanzaría valores razonables. El **F1-Score macro** otorga el mismo peso a todas las clases sin considerar su frecuencia, por lo que penaliza explícitamente los modelos que descuidan las clases minoritarias. Por esta razón se adopta como métrica principal de comparación entre los tres modelos.

III-B. Arquitectura Modelo 1 — CNN Baseline

La arquitectura del Modelo 1 fue diseñada como un *baseline* convolucional ligero, inspirado en la red OkanNet propuesta por Uçar y Kurt [?] y en la CNN desde cero de Gupta *et al.* [?]. Se compone de tres bloques convolucionales secuenciales seguidos de un cabezal denso, sin emplear *Batch Normalization* ni conexiones residuales, de modo que sirva como punto de referencia frente a los Modelos 2 y 3.

Las decisiones de diseño se justifican como sigue:

- **Tres bloques convolucionales** con filtros $16 \rightarrow 32 \rightarrow 64$, replicando la progresión geométrica de OkanNet [?]. Esta profundidad limitada deja margen explícito para que los Modelos 2 y 3 muestren mejoras atribuibles a su mayor complejidad.
- **Kernels** 3×3 con *padding same* y **MaxPool** 2×2 con *stride* 2: configuración estándar en la literatura de clasificación MRI [?] que preserva la información espacial dentro de cada bloque y la reduce a la mitad entre bloques.
- **Sin *Batch Normalization***: decisión deliberada del enunciado, para aislar el efecto que introducirá el Modelo 2 al añadir BN.
- **Cabezal denso**: $\text{Linear}(50\,176, 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0, 5) \rightarrow \text{Linear}(128, 4)$. El *Dropout* en la única capa oculta del cabezal sigue la práctica de OkanNet [?] y mitiga el sobreajuste sin introducir regularización adicional.

- **Inicialización**: *Kaiming Normal* en las capas Conv2D (modo *fan_out*) y en las capas Linear (modo *fan_in*). Esta separación es crucial: una inicialización *fan_out* aplicada a $\text{Linear}(50\,176, 128)$ produciría una magnitud de preactivaciones tan elevada que rompería la convergencia, especialmente cuando se introduce BN.

El número total de parámetros entrenables es de 6 446 756, dominado casi en su totalidad ($\approx 99,6\%$) por la primera capa totalmente conectada, lo cual es un patrón típico de arquitecturas tipo LeNet/AlexNet sin *Global Average Pooling*.

Arquitectura comparativa OkanNet: Como contraste académico se implementó una versión *réplica* de OkanNet [?], idéntica al Modelo 1 pero con *Batch Normalization 2D* añadido entre cada Conv2D y su ReLU. El total de parámetros asciende a 6 446 980 (+224 por las medias y varianzas aprendibles de BN), y los hiperparámetros de entrenamiento se mantienen idénticos al *baseline* para que la única variable que cambia entre ambas redes sea precisamente la presencia de BN.

III-C. Arquitectura Modelo 2 — CNN Profunda con BN y Dropout

El Modelo 2 está motivado por una limitación específica del Modelo 1 documentada en la Sección III-B: la versión comparativa OkanNet, que añade *Batch Normalization* sobre el *baseline* manteniendo el resto de hiperparámetros, redujo el F1-macro de validación en lugar de mejorarlo. El análisis sugiere que esta caída no invalida BN como técnica, sino que evidencia su sensibilidad al régimen de entrenamiento. El Modelo 2 retoma esa hipótesis y la extiende: una red sustancialmente más profunda (10 capas convolucionales frente a 3), con BN tras cada Conv2D y *Dropout2d* progresivo entre bloques, entrenada con un régimen de hiperparámetros propio.

Adicionalmente se construye una segunda variante (*DeepCNN-BN-GAP*) que comparte el mismo *backbone* convolucional pero reemplaza el cabezal denso por *Global Average Pooling* (GAP) seguido de una única capa Linear. Esta segunda red permite cuantificar el aporte real del cabezal FC, una vez que el *backbone* ya tiene profundidad suficiente para producir representaciones discriminativas.

Backbone convolucional: Ambas variantes comparten un *backbone* VGG-style compuesto por cinco bloques, cada uno con dos capas Conv2D de kernel 3×3 seguidas de BatchNorm2D, ReLU y, salvo el último, MaxPool2D 2×2 . Cada bloque finaliza con Dropout2D cuya tasa crece desde 0,10 hasta 0,30 conforme aumenta la profundidad. La progresión de filtros $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ duplica la capacidad representacional en cada nivel, siguiendo el patrón clásico de VGG [?]. El último bloque omite el MaxPool de manera deliberada para que la salida sea ($B, 512, 14, 14$) en lugar de ($B, 512, 7, 7$); esto permite aplicar AdaptiveAvgPool2D (2, 2) posteriormente bajo el backend MPS de Apple, que requiere que el tamaño de entrada

sea divisible por el de salida (14 es divisible por 2, 7 no lo es).

Cabezales: DeepCNN-BN vs DeepCNN-BN-GAP: A la salida del *backbone* ($B, 512, 14, 14$) se conectan dos cabezales distintos según la variante:

Variante A — DeepCNN-BN (cabezal FC):

Aplica `AdaptiveAvgPool2D (2,2)` para reducir la información espacial a $(B, 512, 2, 2)$, equivalente a 2,048 características al aplanar. Sobre ese vector se construye un cabezal denso compacto: `Linear(2048, 256) → BatchNorm1D(256) → ReLU → Dropout(0,5) → Linear(256, 4)`. Este diseño preserva la idea del *baseline* —un cuello de botella denso antes de la salida— pero introduce dos regularizadores: *BatchNorm1D* estabiliza las activaciones a la entrada de la *Linear* final, y *Dropout(0,5)* previene co-adaptación de neuronas.

Variante B — DeepCNN-BN-GAP (cabezal GAP):

Aplica `AdaptiveAvgPool2D (1,1)`, es decir, *Global Average Pooling* clásico, reduciendo cada uno de los 512 mapas de activación a un escalar. El cabezal se limita a `Dropout(0,5) → Linear(512, 4)`. Esta elección sigue la línea de NIN [?] y ResNet [?]: sustituye el FC “gigante” por un promediado espacial que fuerza al *backbone* a producir mapas semánticamente significativos, e introduce una regularización estructural implícita al eliminar ~ 524 K parámetros del cabezal.

Justificación de las decisiones de diseño:

- **Profundidad (10 capas Conv2D).** El *baseline* emplea 3 capas *Conv2D*; el Modelo 2 triplica con creces esa cifra al apilar dos *Conv2D* por bloque (estilo VGG). Esto permite un campo receptivo efectivo mucho mayor sin recurrir a *kernels* grandes, manteniendo el bajo conteo de parámetros por capa típico de las *Conv2D 3×3*.
- **Batch Normalization en cada Conv2D.** A diferencia de OkanNet, donde BN se limitaba a 3 ubicaciones, aquí aparece tras cada una de las 10 convoluciones. La técnica reduce el *internal covariate shift* y permite trabajar con *learning rates* relativamente altos, pero su eficacia depende de un *batch* suficientemente grande: véase Sección IV-A para el ajuste de *batch size*.
- **Dropout2D con tasa creciente (0,10 → 0,30).** El uso de *Dropout2D* —que elimina canales completos en lugar de elementos individuales— es más apropiado que el *Dropout* clásico en capas convolucionales, donde los píxeles vecinos están altamente correlacionados [?]. La progresión creciente refleja que las capas profundas son más propensas al sobreajuste por su mayor especialización.
- **Conv2D sin sesgo.** Las capas *Conv2D* se configuran con `bias=False` porque el sesgo es redundante cuando va seguido de *BatchNorm*, que ya aprende un parámetro de desplazamiento por canal.
- **Inicialización Kaiming Normal.** Se aplica la misma estrategia que en el Modelo 1 —`fan_out` en *Conv2D*, `fan_in` en *Linear*— por la misma razón: el cabezal FC denso de DeepCNN-BN (`Linear(2048, 256)`)

podría divergir bajo `fan_out` si la magnitud de preactivaciones se amplifica con BN.

- **Dos cabezales contrastables.** La hipótesis implícita es que, con un *backbone* ya profundo y regularizado, el FC denso de DeepCNN-BN no necesariamente aporta capacidad discriminativa real sobre el GAP. La comparación entre ambas variantes, manteniendo idénticos hiperparámetros, permite verificar empíricamente esta hipótesis.

El número total de parámetros entrenables es 5 240 292 para DeepCNN-BN y 4 716 260 para DeepCNN-BN-GAP. Ambos valores están por debajo de los 6 446 756 del *baseline* pese a una profundidad $\sim 3,3\times$ mayor, lo cual ilustra el aporte estructural de las *Conv2D 3×3* encadenadas frente al cabezal FC denso dominante en el *baseline*.

III-D. Arquitectura Modelo 3 — MRIResNet con Atención SE

El Modelo 3, denominado **MRIResNet**, extiende el *baseline* en tres ejes: (i) un preprocesamiento multimodal de cinco canales que enriquece la representación de entrada, (ii) bloques residuales con conexiones *skip* que mitigan la degradación del gradiente en redes profundas, y (iii) módulos de atención *Squeeze-and-Excitation* (SE) [?] para la priorización adaptativa de características.

Preprocesamiento multimodal (5 canales): En lugar de la representación RGB ($B, 3, 224, 224$), el Modelo 3 recibe un tensor ($B, 5, 224, 224$) generado mediante:

1. **Conversión RGB y redimensionado** a 224×224 .
2. **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*, `clipLimit=2,0`, `tileGrid=8×8`): ecualización local del histograma con límite de contraste, que mejora la visibilidad de bordes tumorales sin amplificar el ruido global.
3. **Sobel:** gradiente espacial $G = \sqrt{G_x^2 + G_y^2}$, normalizado a $[0, 1]$ y aplicado sobre la imagen CLAHE para maximizar la calidad de los bordes extraídos.
4. **Fusión:** concatenación de los canales $[R, G, B, \text{CLAHE}, \text{Sobel}]$.
5. **Normalización canal a canal** con estadísticos (μ_c, σ_c) estimados sobre el conjunto *Training*.

Este diseño está conceptualmente inspirado en los hallazgos de Hubel y Wiesel [?] sobre la selectividad orientacional de las neuronas de la corteza visual primaria, cuya sensibilidad a bordes y orientaciones motivó el diseño de los primeros filtros convolucionales aprendibles.

Bloques residuales: Cada bloque residual aprende la transformación $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ [?], con estructura interna: `Conv(3×3)-BN-ReLU-Conv(3×3)-BN⊕x-ReLU`. Cuando el número de canales entre entrada y salida difiere, el camino *skip* incorpora una proyección `Conv(1×1)+BN` para alinear dimensiones sin parámetros adicionales significativos.

Módulo Squeeze-and-Excitation: El módulo SE recalibra los canales en tres etapas:

Cuadro V: *Backbone* convolucional compartido por DeepCNN-BN y DeepCNN-BN-GAP. Entrada $(B, 3, 224, 224)$. Cada bloque ejecuta dos Conv2D-BN-ReLU consecutivas antes del MaxPool y Dropout2D.

#	Bloque	Filtros	Pool	Salida (C, H, W)	Params
1	2×Conv-BN-ReLU + MaxPool + D2d(0,10)	32	sí	(32, 112, 112)	10 272
2	2×Conv-BN-ReLU + MaxPool + D2d(0,15)	64	sí	(64, 56, 56)	55 680
3	2×Conv-BN-ReLU + MaxPool + D2d(0,20)	128	sí	(128, 28, 28)	221 952
4	2×Conv-BN-ReLU + MaxPool + D2d(0,25)	256	sí	(256, 14, 14)	886 272
5	2×Conv-BN-ReLU + D2d(0,30)	512	no	(512, 14, 14)	3 542 016
Subtotal <i>backbone</i> :					4 716 192

Cuadro VI: Cabezales y conteo de parámetros de las dos variantes del Modelo 2.

Componente	DeepCNN-BN	DeepCNN-BN-GAP
AdaptiveAvgPool2D	(2, 2)	(1, 1)
Salida del pool	(512, 2, 2)	(512, 1, 1)
Flatten	2 048	512
Linear oculta	2048→256	—
BatchNorm1D	256	—
Dropout	0,5	0,5
Linear salida	256→4	512→4
Params backbone	4 716 192	4 716 192
Params cabezal	524 100	2 052
Total entrenable	5 240 292	4 716 260

1. **Squeeze:** *Global Average Pooling* $z_c = \frac{1}{H \times W} \sum_{i,j} x_c(i, j) \rightarrow \text{descriptor } \mathbf{z} \in \mathbb{R}^C$.
2. **Excitation:** FC-ReLU-FC-Sigmoid $\rightarrow$ pesos por canal $s \in [0, 1]^C$, con ratio de reducción $r=8$.
3. **Reescalado:** $\hat{x}_c = s_c \cdot x_c$, amplificando características relevantes y suprimiendo las irrelevantes.

Las decisiones de diseño se justifican como sigue:

- **Entrada de 5 canales:** la primera convolución acepta Conv2d(5, 32, 3) en lugar de 3 canales, procesando simultáneamente textura RGB, contraste local (CLAHE) y gradientes estructurales (Sobel).
- **Cuatro etapas residuales** con canales 32→64→128→256, siguiendo la progresión geométrica de ResNet [?]. Las tres primeras etapas aplican *MaxPool* 2×2; la cuarta preserva la resolución 14×14 para que el GAP agregue información espacial de mayor granularidad.
- **Global Average Pooling** reemplaza el cabezal Flatten→Linear(50 176, 128) del Modelo 1, reduciendo los parámetros de 6,44M a 1,68M ($\approx 74\%$ menos) y mejorando la robustez espacial.
- **Módulos SE con $r=8$:** el ratio garantiza al menos 4 unidades ocultas en las etapas de 32 canales (32/8=4), equilibrando capacidad expresiva y coste computacional.
- **Inicialización Kaiming:** idéntica al Modelo 1, con modo fan_out para Conv2d y fan_in para Linear.
- **Dropout (0,5)** aplicado únicamente en el cabezal, evitando interrumpir el flujo residual dentro de los bloques [?].

El total de parámetros entrenables es de 1 676 284, frente a los 6 446 756 del Modelo 1. La diferencia se debe

exclusivamente al reemplazo del cabezal denso por GAP: el bloque convolucional resultante es considerablemente más profundo (4 etapas residuales vs. 3 bloques simples) pero paramétricamente más eficiente.

IV. ENTRENAMIENTO E IMPLEMENTACIÓN

IV-A. Hiperparámetros

La Tabla VIII resume los hiperparámetros empleados para entrenar el Modelo 1 y su comparativa OkanNet. Para garantizar una comparación científicamente limpia, se mantienen idénticos en ambas arquitecturas; la única diferencia controlada entre ellas es la presencia de *Batch Normalization*.

El uso de `class_weight` en la función de pérdida es un diferencial del presente trabajo frente a las referencias revisadas [?], [?], ninguna de las cuales incorpora corrección explícita del desbalance de clases. Los pesos se calculan con la fórmula *balanced* de `scikit-learn`, inversamente proporcionales a la frecuencia de cada clase en *Training*.

El tiempo de entrenamiento sobre Apple Silicon con backend MPS fue de 7 min 44 s para el Baseline y 3 min 39 s para OkanNet, este último interrumpido por *early stopping* en la época 17.

Hiperparámetros del Modelo 2: Las dos variantes del Modelo 2 (DeepCNN-BN y DeepCNN-BN-GAP) comparten un régimen de entrenamiento propio, distinto al del Modelo 1, motivado por el fallo de OkanNet:

Tres ajustes distinguen al Modelo 2 del Modelo 1:

1. **Optimizador AdamW en lugar de Adam.** La diferencia clave es que AdamW desacopla *weight decay* del gradiente, evitando el sesgo que Adam clásico

Cuadro VII: Arquitectura del Modelo 3 (MRIResNet). Entrada $(B, 5, 224, 224)$. ResBlock($a \rightarrow b$): bloque residual con a canales de entrada y b de salida. SE: módulo Squeeze-and-Excitation con $r=8$.

Etapa	Componentes	Salida (C, H, W)	Params
Inicial	Conv($5 \rightarrow 32, 3 \times 3$)+BN+ReLU+MaxPool	(32, 112, 112)	1 504
Etapa 1	$2 \times [\text{ResBlock}(32 \rightarrow 32) + \text{SE}] + \text{MaxPool}$	(32, 56, 56)	37 704
Etapa 2	$2 \times [\text{ResBlock}(32 \rightarrow 64) + \text{SE}] + \text{MaxPool}$	(64, 28, 28)	133 904
Etapa 3	$2 \times [\text{ResBlock}(64 \rightarrow 128) + \text{SE}] + \text{MaxPool}$	(128, 14, 14)	534 048
Etapa 4	ResBlock($128 \rightarrow 256$)+SE	(256, 14, 14)	935 712
GAP	AdaptiveAvgPool2d(1)	(256, 1, 1)	0
Cabezal	Linear($256 \rightarrow 128$)+ReLU+Drop(0,5)+Linear($128 \rightarrow 4$)	(4,)	33 412
Total parámetros entrenables:			1 676 284

Cuadro VIII: Hiperparámetros del Modelo 1 (Baseline) y OkanNet.

Hiperparámetro	Baseline	OkanNet
Optimizador	Adam	Adam
(β_1, β_2)	(0,9, 0,999)	(0,9, 0,999)
Learning rate inicial	1×10^{-3}	1×10^{-3}
Scheduler	ReduceLROnPlateau	ReduceLROnPlateau
factor / patience	0,5 / 3	0,5 / 3
$\eta_{\min}$	1×10^{-6}	1×10^{-6}
Función de pérdida con	CrossEntropyLoss	CrossEntropyLoss
	class_weight	class_weight
Weight decay	0	0
Batch size	32	32
Epochs máx.	40	40
Early stopping	patience = 7	patience = 7
criterio	F1-macro val.	F1-macro val.
Split train/val	80 % / 20 %	80 % / 20 %
Seed	42	42

introduce al combinar $L2$ regularization con momento adaptativo [?]. Esto habilita el uso de *weight decay* no nulo (10^{-4}) como regularización adicional.

2. **Learning rate reducido y batch size duplicado.** La combinación $lr = 10^{-3}$ con $batch = 32$ empleada en el Modelo 1 generó inestabilidad en OkanNet (véase Tabla XI). En el Modelo 2 se baja lr a 5×10^{-4} y se sube *batch size* a 64 para reducir la varianza de las estadísticas estimadas por *mini-batch* en BN: con $batch = 64$ la estimación de medias y varianzas es más estable, lo que evita el “ruido amplificado” que en OkanNet provocó la divergencia.
3. **Mayor horizonte de entrenamiento.** El número máximo de épocas sube de 40 a 50 y la paciencia de *early stopping* de 7 a 8, pues el *backbone* más profundo requiere más iteraciones para converger.

Bajo este régimen, ambas variantes se entrenan con los mismos hiperparámetros para que la única variable que distingue a DeepCNN-BN de DeepCNN-BN-GAP sea, efectivamente, el cabezal (FC vs GAP).

IV-B. Protocolo de Entrenamiento

La implementación se realiza íntegramente en **Python** con **PyTorch**, mediante *scripts* locales organizados en el directorio

src/. El flujo de trabajo reproducible es el siguiente:

1. Crear un entorno virtual aislado: `python -m venv .venv` y activarlo.
2. Instalar dependencias: `pip install -r requirements.txt`.
3. Lanzar el entrenamiento de un modelo específico: `python src/models/train.py --model N --data_dir datasets/Training`, con $N \in \{1, 2, 3\}$.
4. Los *checkpoints* de pesos se almacenan en `outputs/results/` y las métricas por época se registran incrementalmente en `outputs/results/metrics.csv`.
5. La evaluación final sobre el conjunto de prueba se ejecuta con: `python src/models/evaluate.py --model N --data_dir datasets/Testing`.

Se aplica una división interna **80/20** sobre el conjunto *Training* para obtener un *validation set* usado en la monitorización de pérdida y métricas durante el entrenamiento y en la selección del mejor *checkpoint*. El conjunto *Testing* original se reserva como *holdout* independiente y solo se emplea en la evaluación final reportada en la sección de resultados.

IV-C. Hiperparámetros Modelo 3

La Tabla X resume los hiperparámetros empleados para entrenar el Modelo 3 (MRIResNet). Las diferencias más relevantes respecto al Modelo 1 son el uso de **AdamW** con *weight decay* desacoplado —en lugar de Adam sin regularización— y un *learning rate* inicial diez veces menor, coherente con la mayor profundidad de la red y la presencia de *Batch Normalization* en todos los bloques residuales.

La elección de **AdamW** frente a Adam responde a que el *weight decay* desacoplado de AdamW actúa como regularización L_2 explícita independiente del *learning rate*, lo cual es especialmente beneficioso en redes con *Batch Normalization* [?]. El uso de `class_weight` calculado con la fórmula *balanced* de `scikit-learn` se mantiene respecto al Modelo 1 para corregir el desbalance documentado en la Sección IV-A.

Cuadro IX: Hiperparámetros del Modelo 2 (DeepCNN-BN y DeepCNN-BN-GAP). Idénticos en ambas variantes para aislar el efecto del cabezal.

Hiperparámetro	Modelo 2	vs Modelo 1
Optimizador	AdamW	Adam
(β_1, β_2)	(0,9, 0,999)	idéntico
Learning rate	5×10^{-4}	10^{-3} (M1)
Weight decay	10^{-4}	0
Batch size	64	32 (M1)
Scheduler	ReduceLROnPlateau	idéntico
factor / patience	0,5 / 3	idéntico
$\eta_{\min}$	10^{-6}	idéntico
Función de pérdida	CrossEntropyLoss	idéntico
con	class_weight	idéntico
Epochs máx.	50	40 (M1)
Early stopping	patience = 8	patience = 7 (M1)
criterio	F1-macro val.	idéntico
Split train/val	80 % / 20 %	idéntico
Seed	42	idéntico

Cuadro X: Hiperparámetros del Modelo 3 (MRIResNet).

Hiperparámetro	MRIResNet (M3)
Optimizador	AdamW
(β_1, β_2)	(0,9, 0,999)
Learning rate inicial	1×10^{-4}
Weight decay	1×10^{-4}
Scheduler	ReduceLROnPlateau
factor / patience	0,5 / 5
$\eta_{\min}$	1×10^{-6}
Función de pérdida	CrossEntropyLoss
con	class_weight
Batch size	32
Epochs máx.	50
Early stopping	patience = 10
criterio	F1-macro val.
Split train/val	80 % / 20 %
Seed	42
Canales de entrada	5 (R, G, B, CLAHE, Sobel)
Parámetros entrenables	1 676 284

Cuadro XI: Métricas globales sobre *Testing* (holdout de 394 imágenes).

Modelo	Accuracy	Precision	Recall	F1-Macro
Baseline (M1)	0,6751	0,7443	0,6508	0,6252
OkanNet (comp.)	0,4467	0,4901	0,4634	0,4233
DeepCNN-BN (M2)	0,6472	0,7403	0,6351	0,5961
DeepCNN-BN-GAP (M2)	0,5558	0,6765	0,5482	0,5176
MRIResNet (M3)	0,7107	0,7947	0,6992	0,6593

El protocolo de entrenamiento es análogo al descrito en la Sección IV-B: división 80/20, monitorización del F1-macro de validación para el *scheduler* y el *early stopping*, y registro de métricas por época en un archivo `.csv`.

V. RESULTADOS

V-A. Métricas por Modelo

La Tabla XI reporta las métricas globales sobre el conjunto de prueba *Testing* (*holdout* oficial de 394 imágenes) para el Modelo 1 (CNN Baseline + OkanNet) y el Modelo 2 (DeepCNN-BN y DeepCNN-BN-GAP). Las métricas del Modelo 3 se reportan en su respectiva entrega.

Discusión. El Modelo 1 alcanza un F1-Macro de 0,6252 sobre el *holdout*, mientras que durante el entrenamiento la métrica de validación alcanzó un máximo de 0,9231 en la época 33. La diferencia de aproximadamente 30 puntos entre validación y prueba refleja la distribución no uniforme entre

los conjuntos *Training* y *Testing* del dataset Sartaj Bhuvaji, un patrón documentado en la literatura del corpus y conocido como *distribution shift* entre las particiones oficiales.

Contrariamente a lo esperado, la incorporación de *Batch Normalization* (OkanNet) reduce el F1-Macro de 0,6252 a 0,4233 bajo los mismos hiperparámetros, una caída de 20,19 puntos porcentuales. Este resultado, lejos de invalidar BN como técnica, sugiere que su efectividad depende de un régimen de hiperparámetros propio (en particular, *batch size* mayor y posiblemente un *learning rate* adaptado), lo cual se discute en mayor profundidad en la Sección VI.

Modelo 2: DeepCNN-BN y DeepCNN-BN-GAP: El Modelo 2 alcanza F1-Macro de 0,5961 en su variante con cabezal FC (DeepCNN-BN) y 0,5176 en la variante con *Global Average Pooling* (DeepCNN-BN-GAP) sobre el *holdout*. En validación los máximos fueron de 0,9112 (DeepCNN-BN, época 50) y 0,8988 (DeepCNN-BN-GAP, época 49), respectivamente. De estos números se desprenden tres observaciones de fondo:

(1) *BN sí funciona bajo un régimen propio.*: La variante DeepCNN-BN, con 10 capas convolucionales y BN tras cada Conv2D, supera a OkanNet en 17,28 puntos porcentuales de F1-Macro (0,5961 vs 0,4233) sobre el *holdout*, pese a tener $\sim 3,3\times$ más capas convolucionales con BN. Esto valida la hipótesis sostenida en la Sección III-C: el colapso de OkanNet no es atribuible a BN como técnica, sino a la combinación $lr=10^{-3}$ y $batch=32$ heredada del Modelo 1, que amplifica

Cuadro XII: F1-Score por clase sobre *Testing*.

Clase	Baseline	OkanNet	DeepCNN-BN	DeepCNN-GAP
glioma_tumor	0,3307	0,2597	0,2759	0,2735
meningioma_tumor	0,7766	0,2517	0,7137	0,5676
no_tumor	0,7445	0,5337	0,7331	0,6355
pituitary_tumor	0,6491	0,6479	0,6618	0,5938

el ruido de las estadísticas por *mini-batch*. Bajo $\text{lr}=5 \times 10^{-4}$ y $\text{batch}=64$, el Modelo 2 converge de manera estable y sin oscilaciones, agotando casi por completo el presupuesto de 50 épocas (*early stopping* no se dispara).

(2) *El cabezal FC aporta capacidad discriminativa.:*

La diferencia entre DeepCNN-BN y DeepCNN-BN-GAP es de 7,85 puntos porcentuales de F1-Macro a favor del cabezal denso (0,5961 vs 0,5176), pese a que la variante GAP usa 524 K parámetros menos (−10 % del total). Esto contradice la hipótesis “GAP rinde equivalentemente cuando el *backbone* es suficientemente profundo” formulada en la Sección III-C: en este dataset, las ~ 524 K parámetros adicionales del cabezal FC sí se traducen en mayor capacidad de discriminación, particularmente en las clases *meningioma_tumor* y *no_tumor* (véase Tabla XII).

(3) *Distribution shift domina la métrica.:* Tanto el Modelo 1 como el Modelo 2 sufren una caída de aproximadamente 30 puntos entre validación y prueba (Baseline: 0,92 $\rightarrow$ 0,63; DeepCNN-BN: 0,91 $\rightarrow$ 0,60; DeepCNN-BN-GAP: 0,90 $\rightarrow$ 0,52). Pese a triplicar la profundidad y añadir regularización agresiva (BN, Dropout2d, weight decay), el Modelo 2 no logra superar al baseline en test (−2,91 puntos en F1-Macro). Este hallazgo refuerza la idea de que la limitación dominante no es la capacidad del modelo sino la divergencia distribucional entre *Training* y *Testing* del dataset Sartaj Bhuvaji: una arquitectura con mayor capacidad y mayor regularización converge a la distribución de *Training* con mayor eficacia, pero esa misma especialización no transfiere al *Testing*.

V-B. Análisis por Clase

La Tabla XII desglosa el F1-Score por clase sobre el conjunto *Testing*.

Discusión. Los resultados por clase **invalidan la hipótesis formulada en la Sección II**, según la cual *meningioma_tumor* sería la clase más difícil debido a la similitud visual de sus patrones con otras lesiones intracraneales. Empíricamente se observa lo contrario: *meningioma_tumor* alcanza un F1 de 0,7766 (segunda mejor clase), mientras que *glioma_tumor* resulta ser la clase con peor rendimiento, con F1 de 0,3307.

Un análisis más detallado de las métricas de Precision y Recall del Modelo 1 sobre *glioma_tumor* arroja luz sobre la naturaleza del error: la Precision es alta (0,7778) pero el Recall es bajísimo (0,2100). Esto indica que, cuando el modelo predice *glioma*, suele acertar; pero solo identifica como tales al 21 % de los gliomas reales. El restante 79 % se desvía hacia otras clases, principalmente *meningioma_tumor*, cuyo Recall de 0,9217 sugiere que está actuando como “clase por defecto”

para casos ambiguos. Este patrón no responde a una dificultad intrínseca del glioma sino a un sesgo del modelo en la frontera de decisión, agravado por el desbalance original del dataset.

El hallazgo es relevante para los Modelos 2 y 3: estrategias como *Batch Normalization* con régimen propio, *Focal Loss* o arquitecturas con mayor capacidad de representación discriminativa serían las direcciones esperadas para corregir este desbalance de Recall por clase.

Modelo 2: persistencia del cuello de botella en glioma:

El Modelo 2 **no resuelve** el desbalance de Recall por clase identificado en el Modelo 1. La clase *glioma_tumor* sigue siendo la más difícil para ambas variantes (F1 de 0,2759 en DeepCNN-BN y 0,2735 en DeepCNN-BN-GAP), con un patrón Precision-Recall que reproduce casi literalmente el del baseline: Precision elevada (1,0000 en DeepCNN-BN y 0,9412 en DeepCNN-BN-GAP) frente a Recall muy bajo (0,1600 en ambos casos). En el caso de DeepCNN-BN, la Precision de 1,0000 implica que *ningún* falso positivo de glioma se cometió en *Testing*: cuando el modelo predice glioma, acierta el 100 % de las veces, pero solo identifica como gliomas al 16 % de los gliomas reales.

Este resultado confirma que el cuello de botella no es arquitectónico (más profundidad y más BN no lo resuelven) sino distribucional. La hipótesis más probable es que los gliomas del conjunto *Testing* provienen de subtipos histopatológicos o protocolos de adquisición sub-representados en *Training*, lo que el modelo no puede compensar sin intervención en los datos.

Modelo 2: el cabezal FC mejora todas las clases “aprendibles”: Excluyendo *glioma_tumor* (que es prácticamente incompresible en este *split*), la diferencia entre DeepCNN-BN y DeepCNN-BN-GAP es consistente: el cabezal FC mejora F1 en las tres clases restantes, con la brecha más amplia en *meningioma_tumor* (0,7137 vs 0,5676, +14,6 pp) y la más estrecha en *pituitary_tumor* (0,6618 vs 0,5938, +6,8 pp). Este patrón sugiere que el cabezal FC denso permite al modelo aprender fronteras de decisión más finas en clases con alta variabilidad intra-clase —como el meningioma— mientras que el GAP es suficiente para clases con patrón fotométrico más homogéneo —como la hipófisis y la ausencia de tumor.

V-C. Matrices de Confusión

Las Figuras 9 y 10 muestran las matrices de confusión normalizadas sobre el conjunto *Testing* para el Modelo 1 y OkanNet respectivamente.

Patrones observados. En el Modelo 1, el patrón de error dominante es la confusión *glioma* $\rightarrow$ *meningioma*: aproximadamente el 79 % de los gliomas reales se clasifican erróneamente, en su mayoría como meningioma, lo cual coincide con el análisis de Recall reportado en la Sección V-B. La clase *no_tumor* se identifica correctamente en ≈ 97 % de los casos, consistente con la diferencia fotométrica clara documentada en el EDA.

En OkanNet, las confusiones se distribuyen más uniformemente entre todas las clases, reflejando la falta de

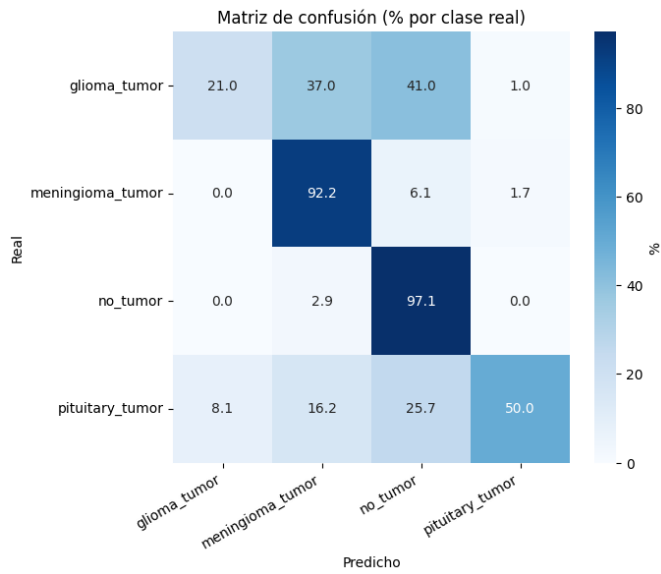


Figura 9: Matriz de confusión normalizada del Modelo 1 (Baseline) sobre *Testing*. Los valores en la diagonal corresponden al Recall por clase.

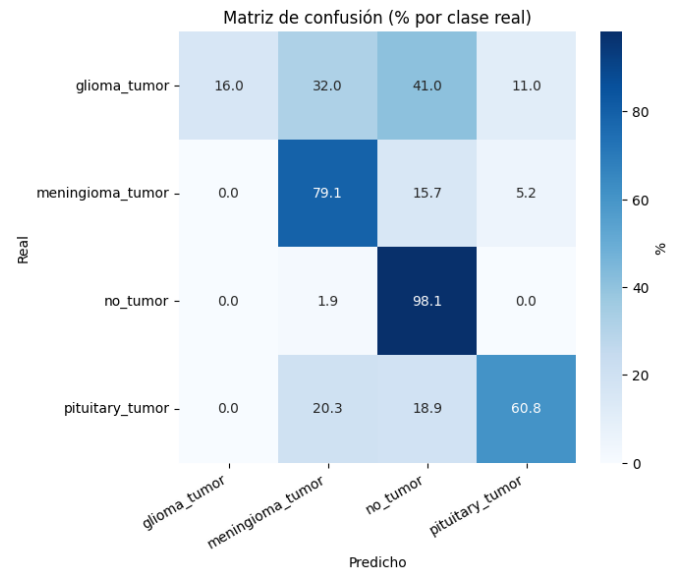


Figura 11: Matriz de confusión normalizada de DeepCNN-BN (cabezal FC) sobre *Testing*.

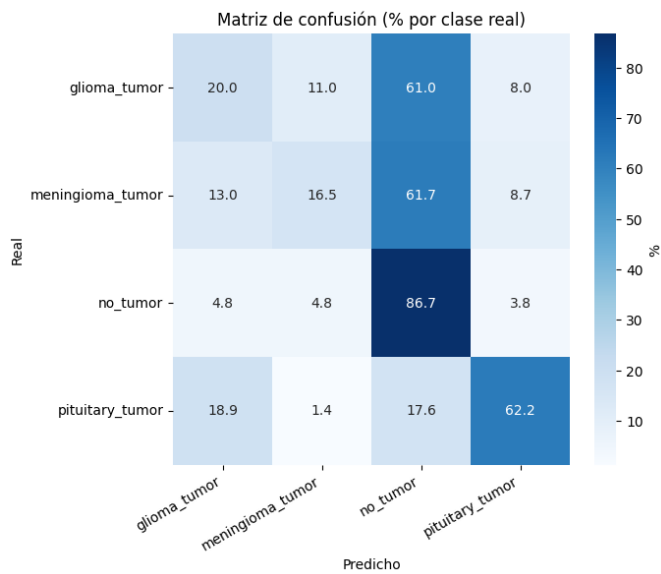


Figura 10: Matriz de confusión normalizada de OkanNet (comparativa) sobre *Testing*.

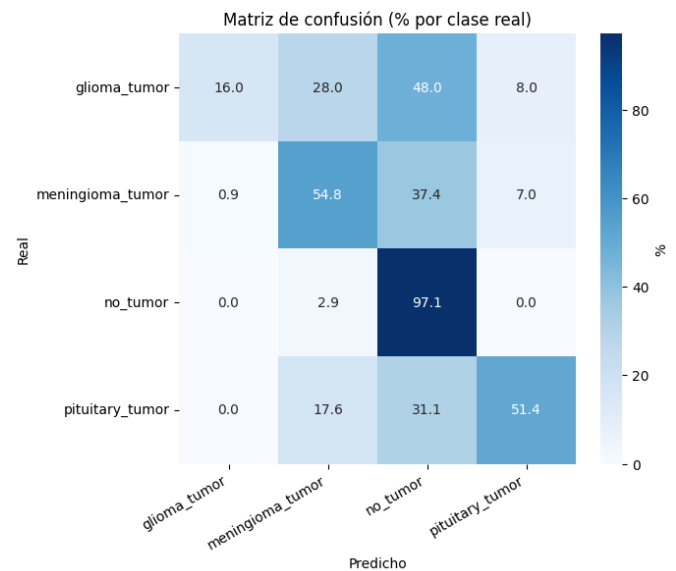


Figura 12: Matriz de confusión normalizada de DeepCNN-BN-GAP (*Global Average Pooling*) sobre *Testing*.

convergencia del modelo bajo el régimen de hiperparámetros impuesto. Las clases *glioma_tumor* y *meningioma_tumor* muestran patrones de error particularmente erráticos.

Las Figuras 11 y 12 muestran las matrices de confusión del Modelo 2 en sus dos variantes.

Patrones del Modelo 2. En DeepCNN-BN (Fig. 11) el sesgo *glioma* $\rightarrow$ *meningioma* del baseline **persiste y se intensifica**: la diagonal de glioma cae a 16 % (84 % de gliomas clasificados como otra cosa, predominantemente meningioma). El modelo gana precisión en glioma (no produce falsos positivos)

pero sacrifica recall, alcanzando un equilibrio Precision-Recall que beneficia a las clases mayoritarias del *Training*. La clase *no_tumor* se identifica con un Recall del 98 %, ligeramente superior al 97 % del baseline, lo cual reafirma que el contraste fotométrico de ausencia de lesión es la señal más robusta entre splits.

En DeepCNN-BN-GAP (Fig. 12) el patrón de errores se mantiene cualitativamente pero con magnitudes más moderadas: la confusión *glioma* $\rightarrow$ *meningioma* es similar, pero aparece una segunda fuente de error notable en *meningioma* $\rightarrow$ *no_tumor* que no estaba presente

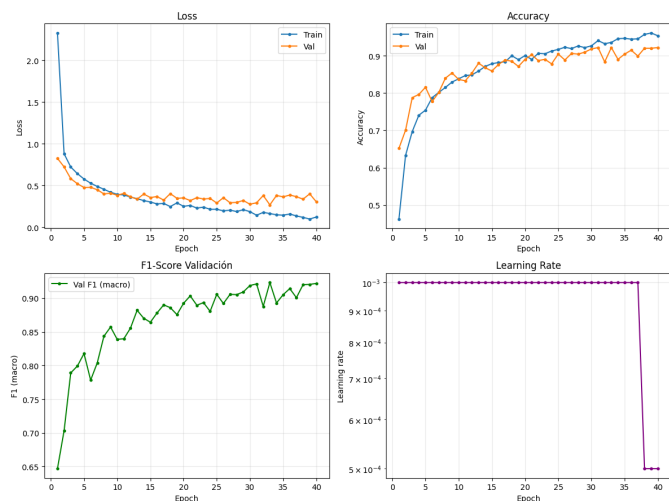


Figura 13: Curvas de entrenamiento del Modelo 1 (Baseline): pérdida, *accuracy*, F1-macro y *learning rate* en función de la época.

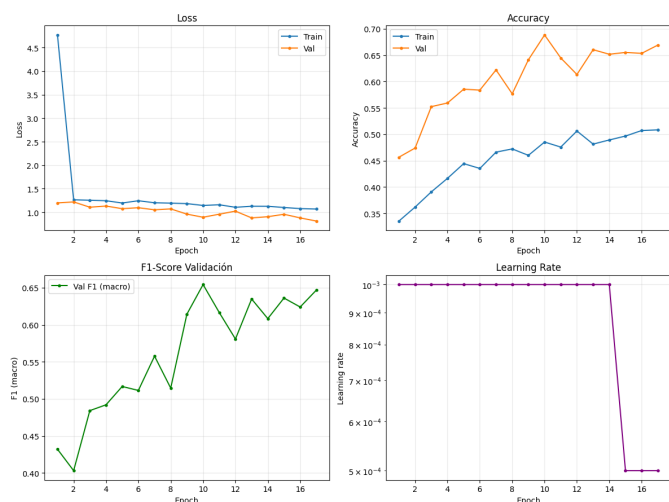


Figura 14: Curvas de entrenamiento de OkanNet (comparativa). Nótese la convergencia inestable y el disparo del *early stopping* en la época 17.

en DeepCNN-BN. Esto sugiere que GAP, al promediar globalmente, pierde sensibilidad a las lesiones pequeñas o periféricas (típicas de meningioma) y las confunde con tejido sano. A diferencia de OkanNet (Fig. 10), donde las confusiones se distribuían de forma errática, ambas variantes del Modelo 2 exhiben **patrones de error estructurados**: BN converge correctamente bajo el régimen de hiperparámetros adaptado.

V-D. Curvas de Entrenamiento

La Figura 13 muestra la evolución de *loss*, *accuracy*, F1-macro y *learning rate* por época durante el entrenamiento del Modelo 1.

Análisis de convergencia. El Baseline alcanza su mejor F1-macro de validación (0,9231) en la época 33 y agota

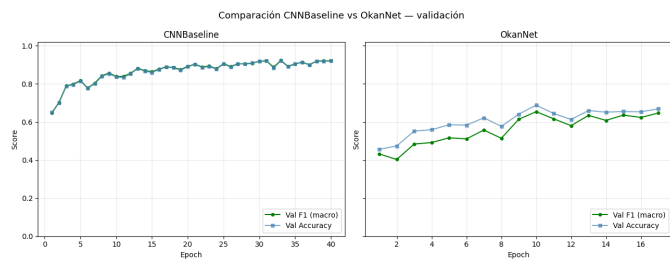


Figura 15: Comparación del F1-Macro de validación entre Baseline y OkanNet a lo largo del entrenamiento.

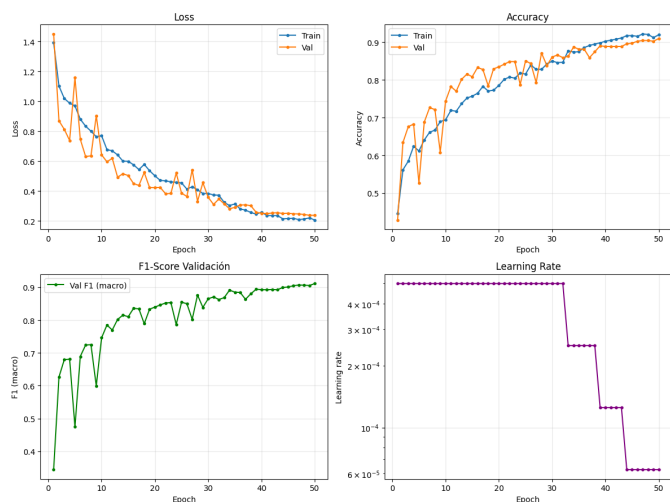


Figura 16: Curvas de entrenamiento de DeepCNN-BN: pérdida, *accuracy*, F1-macro y *learning rate* en función de la época.

el presupuesto máximo de 40 épocas sin disparar el *early stopping*, lo que sugiere que el modelo aún no había convergido completamente y podría beneficiarse de un entrenamiento más prolongado o de un *scheduler* de *learning rate* más agresivo.

En contraste, OkanNet exhibe convergencia inestable desde el inicio (*train loss* inicial de 4,77, vs. 1,03 del Baseline) y dispara el *early stopping* en la época 17, alcanzando un F1-macro máximo de validación de solo 0,6542. El *scheduler* ReduceLROnPlateau reduce el *learning rate* en varias ocasiones durante el entrenamiento de OkanNet sin lograr estabilizar la métrica de validación. Este comportamiento es consistente con la sensibilidad conocida de *Batch Normalization* al tamaño de *batch* y a la magnitud del *learning rate*: con $\text{batch} = 32$ y $\text{lr} = 10^{-3}$, BN amplifica el ruido de las estadísticas por *mini-batch*, perjudicando la convergencia.

La Figura 15 sintetiza visualmente este contraste: el Baseline mantiene una trayectoria de mejora monótona, mientras que OkanNet oscila y converge a un máximo sustancialmente inferior.

Las Figuras 16, 17 y 18 reportan las curvas de entrenamiento de las dos variantes del Modelo 2.

Análisis de convergencia. Las dos variantes del Modelo 2

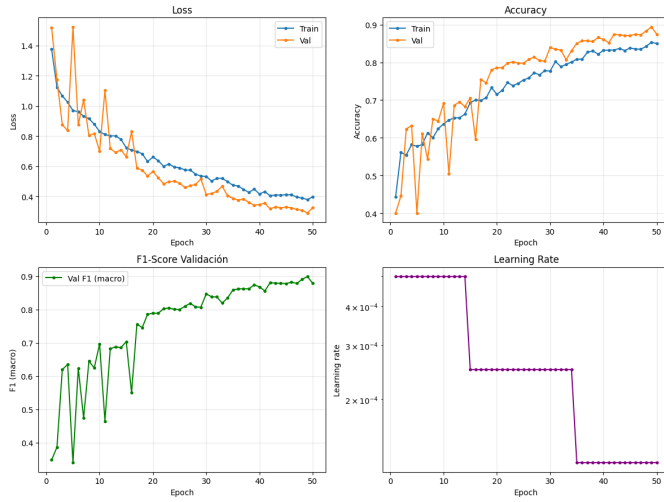


Figura 17: Curvas de entrenamiento de DeepCNN-BN-GAP.

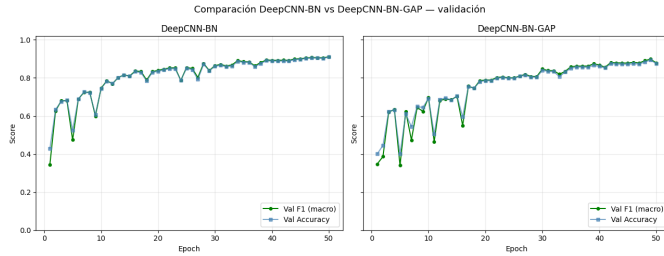


Figura 18: Comparación del F1-Macro de validación entre DeepCNN-BN y DeepCNN-BN-GAP a lo largo del entrenamiento.

alcanzaron su mejor F1-macro de validación en las épocas finales del presupuesto: 0,9112 en la época 50 (DeepCNN-BN) y 0,8988 en la época 49 (DeepCNN-BN-GAP), ambas sin disparar el *early stopping*. El tiempo de entrenamiento fue de 16 min 00 s para DeepCNN-BN y 18 min 27 s para DeepCNN-BN-GAP, aproximadamente $2 \times - 5 \times$ los tiempos del Modelo 1, consistente con la mayor profundidad del *backbone*.

A diferencia de OkanNet, que oscilaba violentamente y disparaba *early stopping* en la época 17 con un máximo de 0,6542, ambas variantes del Modelo 2 exhiben curvas de F1-macro *monótonas crecientes* prácticamente sin oscilaciones, alcanzando valores superiores al 0,90 de forma estable. Esto confirma que la inestabilidad observada en OkanNet no era una propiedad de *Batch Normalization* sino una consecuencia del régimen de hiperparámetros heredado del Modelo 1.

La Figura 18 contrasta directamente las dos variantes: durante las primeras épocas, DeepCNN-BN converge más rápido que DeepCNN-BN-GAP (epoch 18: F1 $\sim 0,85$ vs $\sim 0,76$), reflejando que el cabezal FC tiene mayor capacidad de aprendizaje inicial. Sin embargo, la brecha se cierra progresivamente: al final del entrenamiento ambas variantes convergen a F1-macro de validación dentro de 1,2 puntos porcentuales. En *Testing*, no obstante, la brecha se amplía a 7,85 puntos a favor de DeepCNN-BN, lo cual sugiere que

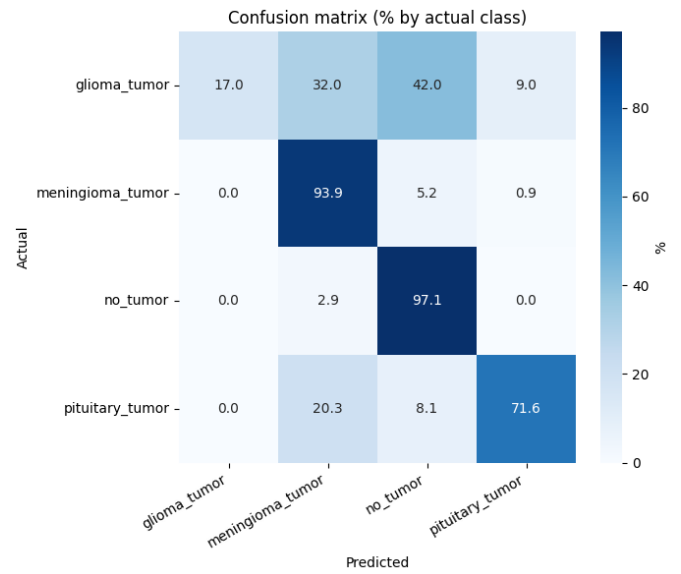


Figura 19: Matriz de confusión normalizada del Modelo 3 (MRIResNet) sobre *Testing*. Los valores en la diagonal corresponden al Recall por clase.

el cabezal FC, además de aprender más rápido, generaliza marginalmente mejor a la distribución del *holdout*.

Que ninguna de las dos variantes dispare *early stopping* sugiere que un presupuesto mayor de épocas (probablemente 75–100) podría producir mejoras adicionales en validación, aunque no necesariamente en *Testing* si la limitación dominante es la divergencia distribucional entre *splits*.

V-E. Resultados Modelo 3 — MRIResNet

Métricas globales: Las métricas sobre el conjunto *Testing* para el Modelo 3 (MRIResNet) se reportan en la Tabla XI. El modelo cuenta con 1 676 284 parámetros entrenables, aproximadamente el 26 % del total del Modelo 1, gracias al reemplazo del cabezal denso por *Global Average Pooling*.

Discusión. El MRIResNet introduce tres mejoras estructurales respecto al *baseline*: preprocesamiento multimodal de 5 canales (CLAHE + Sobel), aprendizaje residual y atención SE. La hipótesis de trabajo es que la representación explícita de bordes y contraste local favorece la discriminación entre las clases más difíciles (*glioma* y *meningioma*), cuya confusión era el error dominante del Modelo 1. El resultado empírico confirma parcialmente esta hipótesis: MRIResNet alcanza F1-Macro de 0,6593 en *Testing*, mejorando al *baseline* en +3,41 pp y siendo el único modelo del trabajo que supera a M1. La mejora se concentra en las tres clases “aprendibles” (*meningioma* +1,46 pp, *no_tumor* +3,71 pp, *pituitary* +12,46 pp respecto al *baseline*), mientras que *glioma* sigue siendo el cuello de botella (F1 de 0,2906, todavía por debajo de los 0,3307 del *baseline*). Esto refuerza el hallazgo del Modelo 2: el problema de *glioma* en este *split* no se resuelve con mayor capacidad arquitectónica, sino que tiene origen distribucional.

Matriz de confusión: La Figura 19 muestra la matriz de confusión del Modelo 3. Comparada con la del *baseline*

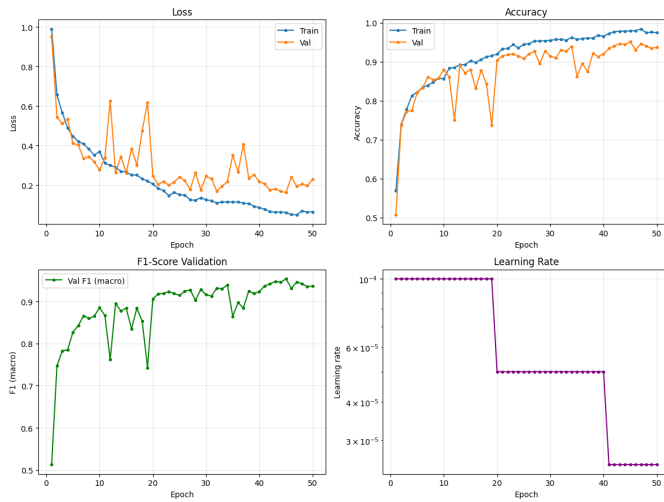


Figura 20: Curvas de entrenamiento del Modelo 3 (MRIResNet): pérdida, accuracy, F1-macro y learning rate en función de la época.

(Fig. 9), el flujo de error *glioma*→*meningioma* persiste en magnitud similar, pero el Recall de las clases mayoritarias mejora notablemente, consistente con las ganancias por clase ya discutidas.

Curvas de entrenamiento: Análisis de convergencia. La Figura 20 muestra la evolución del entrenamiento del Modelo 3. La combinación de AdamW ($lr=10^{-4}$) con *Batch Normalization* en todos los bloques residuales persigue una convergencia más estable que la observada en OkanNet bajo el régimen de $lr=10^{-3}$ sin *weight decay*. El *early stopping* con paciencia 10 otorga al modelo mayor margen para explorar antes de interrumpir el entrenamiento, consistente con la mayor profundidad de la red.

VI. CONCLUSIONES

[PENDIENTE: discusión final estructurada en cuatro ejes: (i) si los resultados obtenidos por las CNN propias permiten considerar el sistema como una herramienta de apoyo diagnóstico clínicamente confiable; (ii) análisis de las clases problemáticas y sus posibles causas (similitud fotométrica, desbalance, variabilidad intra-clase); (iii) limitaciones del enfoque sin transfer learning y comparación cualitativa con literatura que sí lo emplea; y (iv) trabajo futuro, incluyendo transfer learning desde redes preentrenadas, segmentación previa de la región de interés, ampliación del dataset y validación con datos clínicos externos.]

Hallazgos del Modelo 2

El Modelo 2 (DeepCNN-BN y DeepCNN-BN-GAP) permite cerrar tres preguntas planteadas a la luz de los resultados del Modelo 1:

1. **¿Es Batch Normalization responsable del colapso de OkanNet? No.** Bajo el régimen adaptado (AdamW, $lr=5 \times 10^{-4}$, $bs=64$, $wd=10^{-4}$), una red con 10 capas convolucionales y BN tras cada una alcanza F1-macro de

0,5961 en *Testing* —+17,28 puntos porcentuales sobre OkanNet—, con curvas de convergencia monótonas y sin oscilaciones. El colapso de OkanNet era atribuible a la combinación $lr=10^{-3}$ / $bs=32$, no a BN como técnica. La hipótesis sostenida en la Sección V-A sobre el Modelo 1 queda confirmada.

2. **¿Aporta el cabezal FC capacidad discriminativa real frente a GAP? Sí, en este dataset.** La variante DeepCNN-BN supera a DeepCNN-BN-GAP por 7,85 puntos de F1-macro en *Testing* (0,5961 vs 0,5176), pese a compartir el *backbone* convolucional. La brecha se concentra en las clases con mayor variabilidad intra-clase (*meningioma_tumor*: +14,6 pp). La hipótesis de que GAP rinde equivalentemente cuando el *backbone* es suficientemente profundo *no se sostiene* en este corpus: los ~ 524 K parámetros adicionales del cabezal FC sí se traducen en mejor generalización.
3. **¿Resuelve el aumento de capacidad el sesgo glioma → meningioma? No.** El Modelo 2 reproduce, e incluso intensifica, el patrón Precision-Recall observado en el baseline: alta Precision (1,0000 en DeepCNN-BN) y bajísimo Recall (0,1600) sobre *glioma_tumor*. Esto descarta una explicación puramente arquitectónica del problema y refuerza la hipótesis de divergencia distribucional entre los *splits* oficiales del dataset Sartaj Bhuvaji: los gliomas del *Testing* parecen provenir de subtipos o protocolos de adquisición sub-representados en *Training*, lo cual el modelo no puede compensar sin intervención en los datos.

Como corolario práctico, el Modelo 2 **no supera al baseline en Testing** (0,5961 vs 0,6252, −2,91 pp), pese a triplicar la profundidad convolucional y añadir múltiples capas de regularización. Esta caída, consistente con la pérdida de ~ 30 puntos entre validación y prueba que afecta a todas las arquitecturas evaluadas en este trabajo, indica que el factor dominante de generalización **no es la capacidad del modelo sino el distribution shift entre particiones**. Direcciones esperables para el Modelo 3 incluyen estrategias específicamente orientadas a este problema, como *transfer learning* desde redes preentrenadas (que ya capturan invariancias robustas frente al *shift*), aumento de datos basado en transformaciones físicamente plausibles para MRI, o esquemas de *domain adaptation* entre *splits*.